Trabajo práctico 1

Análisis de Componentes Principales

1. Introducción

En muchas aplicaciones modernas, como el aprendizaje automático, resulta fundamental la forma en que se manipulan y representan los datos. La calidad y la estructura de los mismos impactan directamente en la eficiencia de los algoritmos y en la capacidad de obtener resultados precisos. Cuando los datos son de alta dimensionalidad, procesar todas las variables puede ser costoso en términos de tiempo y recursos, y algunas características pueden resultar redundantes o poco informativas. Una estrategia común para abordar este desafío consiste en reducir la dimensionalidad de los datos, representándolos en un espacio de menor dimensión donde se conserve la mayor parte de la información relevante. Entre las herramientas más utilizadas para este propósito se encuentra el *Análisis de Componentes Principales* (PCA, por sus siglas en inglés). PCA transforma las variables originales en un nuevo conjunto de variables no correlacionadas, llamadas **componentes principales**, que capturan la mayor parte de la variabilidad de los datos. De esta forma, al proyectar los datos sobre las primeras direcciones principales, se concentra la información más importante en un espacio de menor dimensión, facilitando su análisis y posterior uso en aplicaciones como clasificación o compresión.

En este trabajo práctico se explorará el uso de PCA aplicado a la clasificación de imágenes de múltiples clases a partir de un dataset provisto por la cátedra. El objetivo es brindar un marco práctico para comprender cómo PCA permite concentrar propiedades importantes de los datos en un espacio reducido, facilitando su análisis, visualización y uso en algoritmos de clasificación.

1.1. Conceptos básicos sobre Clasificación

La clasificación de datos es el proceso mediante el cual se construye un modelo matemático capaz de predecir a qué categoría pertenece un dato a partir de sus características. Cada categoría representa una población de datos que comparten las mismas propiedades. **Ejemplo:** si se consideran distintas clases de tornillos, cada uno puede describirse mediante variables como peso, diámetro y longitud; el objetivo es determinar a qué tipo pertenece un tornillo en base a la observación de esas características. Uno de los enfoques más utilizados es el de clasificación supervisada, que requiere una etapa de entrenamiento en la que el modelo se ajusta a un conjunto de datos con sus clases previamente etiquetadas, Figura 1-(a). Durante esta etapa, el modelo aprende relaciones entre las características de los datos y sus respectivas clases. Una vez entrenado, el modelo puede aplicarse a nuevos datos cuyas clases son desconocidas, con el fin de predecirlas, Figura 1-(b). Para evaluar la clasificación, los datos disponibles se dividen para formar un conjunto de entrenamiento y otro conjunto para prueba, ya que evaluar sobre los mismos datos de entrenamiento no permitiría medir la capacidad de generalización del modelo. Se utilizará como métrica de desempeño el *accuracy* (proporción de aciertos) definida como:

$$\text{accuracy} = \frac{\text{total de aciertos}}{\text{total de datos evaluados}} \tag{1}$$

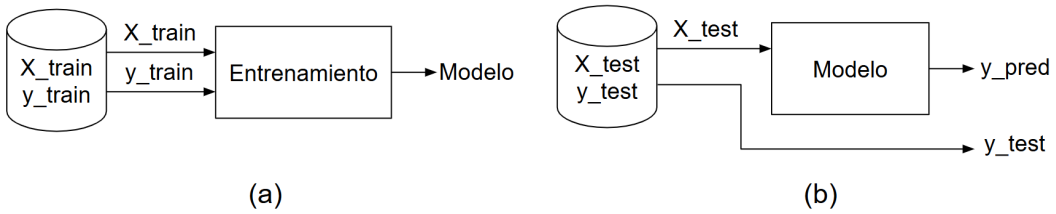

Figura 1: (a) Etapa de entrenamiento; (b) Clasificación usando el modelo entrenado. Cada predicción (`y_pred`) se compara con las etiquetas de prueba `y_test` para calcular el Accuracy.

Existen diversos tipos de modelos de clasificación, entre ellos: *k-nearest neighbors* (k-nn) [1], *Naive Bayes*, *Regresión Logística*, *Redes Neuronales*, *Máquinas de Soporte Vectorial*, etc. En este trabajo práctico, utilizaremos el modelo k-nn, por ser uno de los más simples, sin adentrarnos en sus detalles internos (ya que no es el objetivo del curso), sino utilizándolo como una herramienta para analizar la representación de los datos mediante PCA. En el apéndice se puede ver un código de ejemplo que muestra el uso de un clasificador k-nn, disponible mediante el paquete `sklearn`.

Aclaración: La letra "k" de *k-nearest neighbors*, es un parámetro propio del clasificador, mientas que la "K" que usamos para PCA, es la cantidad de componentes principales (podría ser cualquier otro símbolo). No existe relación alguna entre ambos parámetros, pero para PCA lo mantenemos por consistencia con la notación utilizada en clase.

1.2. Dataset utilizado

Se trabajará con un dataset que contiene 4 clases de imágenes, correspondientes a fotografías de células sanguíneas de distintos tipos: *eosínofilo*, *mieloblasto*, *monocito* y *plaqueta*. Estos datos están extraídos de la base de datos gratuita *MedMNIST* [2]. Para cada clase, se proveen un gran número de imágenes de 28×28 píxeles. En la Figura 2 se ven algunos ejemplos por cada clase:

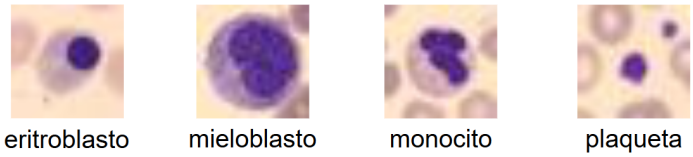

Figura 2

2. Ejercicios

En los siguientes ejercicios se explorará el desempeño de PCA para el conjunto de imágenes del dataset provisto. En términos generales, se buscará, por un lado, evaluar cómo la reducción de dimensionalidad afecta la información útil de una imagen. Luego se analizará el uso de PCA como etapa previa a un clasificador k-nn. En cualquiera de los ejercicios se pueden sumar los gráficos, esquemas o ecuaciones que se consideren pertinentes, adicionalmente a lo pedido en las consignas. Para cada clase, será suficiente trabajar con 500 imágenes elegidas aleatoriamente.

Ejercicio 1

(a) Para analizar cómo se degrada la información de los datos a medida que se eliminan componentes, se aplicará PCA a cada una de las clases por separado. Para cada clase del dataset, reducir la dimensión de los datos a K componentes principales, considerando diferentes valores de este parámetro: $K = 1, 2, 3, 4, 5, 10, 20, 50, 100, 200, 500, 784$. Calcular en cada caso la varianza explicada y el error cuadrático medio (MSE). Graficar las 4 curvas de varianza explicada vs. K para las clases utilizadas. Hacer lo mismo con las curvas de MSE vs. K. Analice los resultados obtenidos.

(b) Para cada clase, elegir una imagen particular y graficar: i) imagen original; ii) imagen reducida y reconstruida con $K = 50$; iii) imagen reducida y reconstruida con $K = 2$. Justifique las diferencias observadas.

Ejercicio 2

(a) Aplicar un modelo de clasificación utilizando los vectores completos, sin reducción de dimensionalidad, como entrada del modelo K-NN, y calcular el *accuracy* sobre el conjunto de prueba.

(b) Aplicar nuevamente el modelo de clasificación, pero esta vez utilizando como vector de entrada los datos reducidos mediante PCA para distintos valores de K (los mismos que en el Ejercicio 1). Calcular y graficar el *accuracy* en función de K. Comparar los resultados con el *accuracy* obtenido sin aplicar PCA. Justifique lo observado.

Ejercicio 3

Realice un gráfico de dispersión en 2D para las 500 imágenes utilizadas por cada clase, considerando únicamente las dos primeras componentes del vector de datos, etiquetando cada clase con un color diferente, en los siguientes dos casos:

(a) Vectores originales de las imágenes truncados a sus dos primeras componentes.

(b) Vectores en el espacio de componentes principales reducidos con PCA para $K = 2$.

Explique y justifique los resultados observados. ¿Cómo impacta esto en la clasificación?

3. Conclusiones

Como conclusiones, elabore un resumen breve y conciso comentando características que considere relevantes del método propuesto en este trabajo y los resultados obtenidos, así como dificultades encontradas y cómo fueron abordadas.

4. Apéndice

En el siguiente código se muestra un ejemplo de cómo definir los conjuntos de entrenamiento y prueba, el entrenamiento del modelo y la validación en la prueba:

```
# Módulos necesarios para clasificación con k-nn
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
...

# Partición de conjuntos train y test (todas las clases juntas)
# Se separan los dos conjuntos, asegurando mezcla aleatoria de las clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)
...

# Definición del modelo de clasificación
clf = KNeighborsClassifier(n_neighbors=5)

# Entrenamiento del modelo
clf.fit(X_train, y_train)

# Validación del modelo
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

Aclaraciones:

- `X`: matriz cuyas filas son las realizaciones de los vectores (de todas las clases juntas).
- `y`: etiquetas de cada clase asociadas a cada vector en `X`.
- `X_train`: matriz cuyas filas son las realizaciones de los vectores del conjunto de entrenamiento (las clases quedan mezcladas aleatoriamente).
- `y_train`: arreglo con las etiquetas de cada clase asociada a cada vector en `X_train`.
- `X_test`: matriz cuyas filas son las realizaciones de los vectores del conjunto de prueba (las clases quedan mezcladas aleatoriamente).
- `y_test`: arreglo con las etiquetas de cada clase asociada a cada vector en `X_test`.
- `y_pred`: arreglo cuyos elementos corresponden con la clasificación de cada vector de prueba (filas de `X_test`).
- `acc` (accuracy): es la proporción de predicciones correctas comparando las etiquetas `y_test` e `y_pred`.

5. Condiciones de entrega

- **Archivo ZIP**: en un archivo comprimido en formato ZIP, de nombre `TP2_GXX.ZIP` (donde `XX` es el número de grupo, ej: `TP2_G01.ZIP`) debe incluirse tanto el informe como el código fuente del trabajo. Este archivo debe subirse al recurso disponible en el campus.
- **Informe**: debe ser en formato PDF (**no se aceptarán otros formatos**) y con nombre: `TP2_GXX.PDF`. Es condición necesaria para la aprobación cumplir con las pautas para la presentación de informes estipuladas en el campus.
- **Código**: el código debe ser en uno o más archivos `.py`, incluyendo un archivo de texto adicional con la lista de paquetes que requieran instalación. **Nota**: No debe incluirse la carpeta de imágenes entre los archivos entregados.
- Se recuerda a los estudiantes que las entregas deben ser un producto original de cada grupo, por lo que se les pide revisar la sección 6 del programa de la materia y el Código de Honor y Ética de la Universidad.

Referencias

[1] GeeksforGeeks, "K Nearest Neighbours (KNN) in Machine Learning," 2024. Disponible en: https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/.

[2] Yang et al., *MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification*, Scientific Data, 2022. Disponible en: https://medmnist.com/